

Concorrência com Threads

Mário Leite

Em programação, **concorrência** é a capacidade de um programa lidar com múltiplas tarefas ao mesmo tempo. Uma das formas mais comuns de implementar concorrência é através de *threads*.

Threads são fluxos de execução dentro de um mesmo processo. Diferentemente de processos independentes, elas compartilham o mesmo espaço de memória, o que facilita a comunicação entre as tarefas, mas também exige cuidado para evitar conflitos de acesso aos dados. Em Python, o módulo *threading* fornece recursos simples e eficientes para criar e controlar **threads**.

No exemplo a de código mostrado ao final desta postagem, implementamos um pequeno programa que executa duas tarefas simultaneamente, demonstrando um modelo simples de multitarefas cooperativas.

Como o programa funciona

O funcionamento do programa pode ser dividido em algumas etapas.

1. Definição das tarefas

As funções **ExecutarTarefa1()** e **ExecutarTarefa2()** representam duas tarefas independentes que serão executadas concorrentemente.

Cada uma executa um laço que imprime uma mensagem cinco vezes.

2. Simulação de tempo de processamento

A instrução: `time.sleep(1)`

Faz com que cada tarefa pause por um segundo, simulando uma operação que leva algum tempo para ser executada. Isto ajuda a visualizar melhor o comportamento concorrente dos *threads*.

3. Criação dos threads

As *threads* são criadas associando uma função a cada thread:

```
tarefa1 = threading.Thread(target=ExecutarTarefa1)
```

```
tarefa2 = threading.Thread(target=ExecutarTarefa2)
```

Cada thread executará a função indicada no parâmetro *target*.

4. Início da execução concorrente

O método **start()** inicia a execução dos threads:

```
tarefa1.start()
```

```
tarefa2.start()
```

A partir desse momento, as duas tarefas passam a executar de forma concorrente.

5. Sincronização com `join()`

O método `join()` faz o programa principal aguardar o término das *threads* antes de continuar:

```
tarefa1.join()
```

```
tarefa2.join()
```

Isso garante que a mensagem final só será exibida depois que as duas tarefas forem concluídas.

Saída típica do programa

A saída pode variar a cada execução, pois os *threads* executam concorrentemente.

Um exemplo possível seria:

Tarefa: 1 executando...

Tarefa: 2 executando...

Tarefa: 1 executando...

Tarefa: 2 executando...

Tarefa: 2 executando...

Tarefa: 1 executando...

Tarefa: 1 executando...

Tarefa: 2 executando...

Tarefa: 1 executando...

Tarefa: 2 executando...

Ambas as tarefas foram concluídas.

Observe que as mensagens das duas tarefas se intercalam, mostrando que ambas estão sendo executadas simultaneamente.

Conclusão

O uso de *threads* permite que um programa execute múltiplas tarefas de maneira concorrente dentro de um mesmo processo.

O módulo *threading* de Python fornece uma forma simples de implementar esse modelo de execução, sendo muito útil em aplicações que precisam lidar com:

- múltiplas tarefas independentes
- operações de entrada e saída
- processamento simultâneo de atividades

Esse exemplo simples demonstra os conceitos básicos de criação, execução e sincronização de threads. A **figura 1** mostra a saída do programa “**UsandoConcorrencia**”.

```

'''
UsandoConcorrencia.py
-----

Neste programa o módulo threading fornece uma maneira de criar/gerenciar threads
de maneira bem simples e objetiva.
-----

'''

import threading
import time

#-----
# Tarefa 1
def ExecutarTarefa1():
    for _ in range(5):
        print("Tarefa 1 executando...")
        time.sleep(1)

#-----
# Tarefa 2
def ExecutarTarefa2():
    for _ in range(5):
        print("Tarefa 2 executando...")
        time.sleep(1)

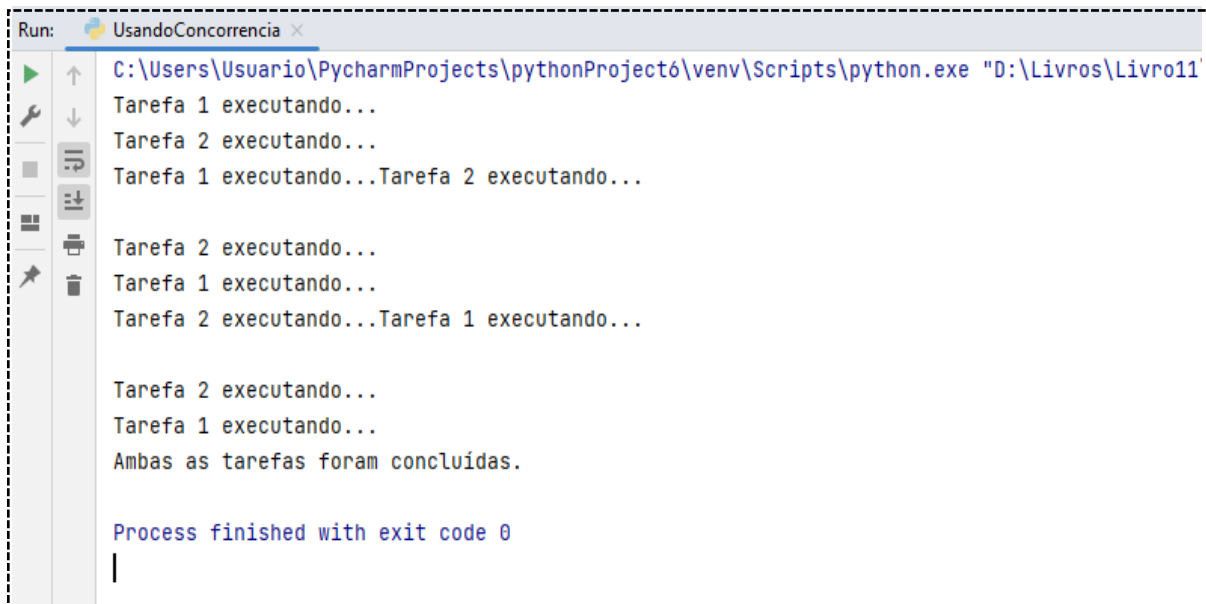
#=====
# Programa principal
# Criação das threads
tarefa1 = threading.Thread(target=ExecutarTarefa1)
tarefa2 = threading.Thread(target=ExecutarTarefa2)

# Início da execução concorrente
tarefa1.start()
tarefa2.start()

# Aguarda o término das duas threads
tarefa1.join()
tarefa2.join()

print("Ambas as tarefas foram concluídas.")
#=====
# Fim do programa "UsandoConcorrencia1"-----

```



The screenshot shows a PyCharm Run console window titled "UsandoConcorrencia". The console output displays the execution of a program that runs two tasks concurrently. The output lines are as follows:

```
C:\Users\Usuario\PycharmProjects\pythonProject6\venv\Scripts\python.exe "D:\Livros\Livro11"
Tarefa 1 executando...
Tarefa 2 executando...
Tarefa 1 executando...Tarefa 2 executando...

Tarefa 2 executando...
Tarefa 1 executando...
Tarefa 2 executando...Tarefa 1 executando...

Tarefa 2 executando...
Tarefa 1 executando...
Ambas as tarefas foram concluídas.

Process finished with exit code 0
|
```

Figura 1 - Saída do programa "UsandoConcorrencia"